

Lab Assignment 6

CMPE 146 Section 1

Professor Bhawandeep Singh Harsh

Andrew Bertch

Robert Trinh

Table of Contents

Exercise 1 – UART	3
Exercise 1.1 – Echo.....	3
Exercise 1.2 – Send text string	4
Exercise 1.3 – Emulate UART	5
Exercise 2 – EnergyTrace	9
Exercise 2.1 – Simple Checksum.....	9
Exercise 2.2 – Shutdown LPM.....	11
Appendix A – MCU Source Code.....	12

Exercise 1 – UART

In this exercise, we will communicate with the LaunchPad from our laptops over the UART protocol. After downloading and installing PuTTY, we can launch a terminal window to monitor COM5 at 115200 baud to see the output from the MCU in the following exercises.

Exercise 1.1 – Echo

In this exercise, we will echo what is input on the laptop keyboard in the PuTTY terminal back to the terminal. When the program is not running, when typing in the terminal nothing will appear. However, with the program running on the MCU, when we type, the text will now appear on the terminal window. This can be seen below in a screenshot of the PuTTY terminal.

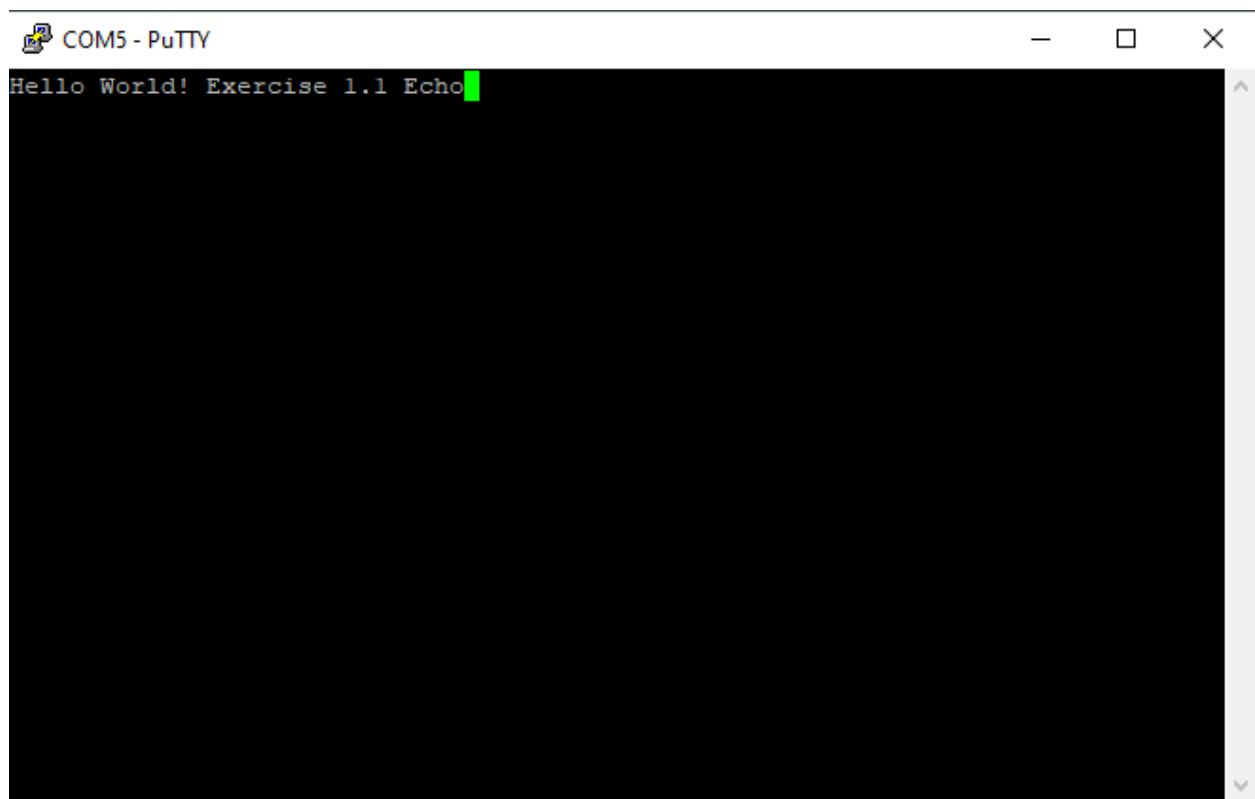


Figure 1.1

The control loop for the echoing is shown below. The whole control loop sits inside the forever while loop with an if statement to check if the receive buffer is not empty. If it isn't empty it will then transmit what is in the receive buffer.

```
while (1) {  
    if (!DL_UART_isRXFIFOEmpty(UART0)) {  
        DL_UART_Main_transmitData(UART0,  
DL_UART_Main_receiveData(UART0));  
    }  
}
```

Exercise 1.2 – Send text string

In this exercise, we will expand upon the echo exercise to first send a function to transmit a character string before starting the echo. The character string is null terminated so the function must look for the last character in the string, which is denoted by a 0x00 char. The function to send a message is included below and uses similar logic to the echo with some minor tweaks. The while loop will continue as long as the current char isn't 0x00, and will transmit the data as long as the transmission buffer isn't full.

```
void send_message(char* msg) {  
    int i = 0;  
    while (msg[i] != 0x00) {  
        if (!DL_UART_isTXFIFOFull(UART0)) {  
            DL_UART_Main_transmitData(UART0, msg[i]);  
            i++;  
        }  
    }  
}
```

A screenshot of the terminal window with the message and echo is included below.

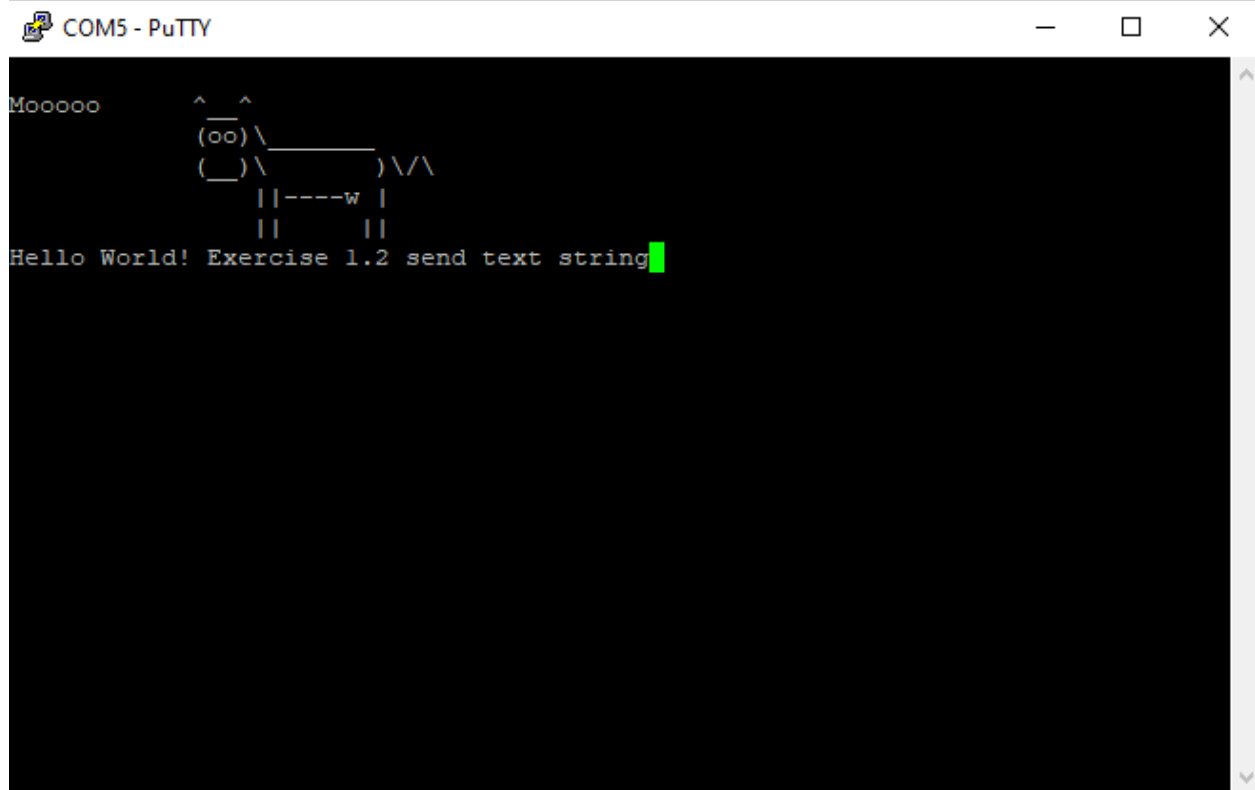


Figure 1.2

Exercise 1.3 – Emulate UART

In this exercise, we will be bypassing the UART controller and emulating UART communication by sending and managing each of our data bits ourselves. We will have to send our data on the correct GPIO pin, PA10, and generate a UART frame with start and stop bits. We also will focus on only the `send_message()` function and will not implement the echo functionality. The `send_message()` function is shown below.

```
void send_message(char* msg) {  
    int count = 0;  
  
    DL_GPIO_enablePower(GPIOA);  
    DL_GPIO_initDigitalOutput(IOMUX_PINCM21);    //UART TX  
    DL_GPIO_enableOutput(GPIOA, DL_GPIO_PIN_10);  
    DL_GPIO_initDigitalInput(IOMUX_PINCM22);    //UART RX
```

```

DL_GPIO_setPins(GPIOA, DL_GPIO_PIN_10);
delay_cycles(3200000);

while (msg[count] != 0x00) {
    char data = msg[count];

    DL_GPIO_clearPins(GPIOA, DL_GPIO_PIN_10);
    delay_cycles((CPUCLK_FREQ / BAUD_RATE));
    for (int i = 0; i < 8; i++) {
        if (data & 1) {
            DL_GPIO_setPins(GPIOA, DL_GPIO_PIN_10);
            delay_cycles(CPUCLK_FREQ / BAUD_RATE);
        }
        else {
            DL_GPIO_clearPins(GPIOA, DL_GPIO_PIN_10);
            delay_cycles(CPUCLK_FREQ / BAUD_RATE);
        }
        data >>= 1;
    }
    DL_GPIO_setPins(GPIOA, DL_GPIO_PIN_10);
    delay_cycles(CPUCLK_FREQ / BAUD_RATE);
    count++;
}
}

```

The function begins with initial pin setup and delays for a set time to establish the idle state for the UART communication. The while loop from the previous send_message() loop is used again and the start bit sent. All delays are calculated using the defined BAUD_RATE global that divides the CPUCLK_FREQ global that TI's firmware keeps

updated. Since we are sending char sized data, we must send 8 bits in each data segment of the UART frame. This can be done with a right shift of 1 after each bit has been sent and logical AND to check if a 1 or 0 should be sent.

Included below is a screenshot of the PuTTY terminal with the program using a baud rate of 9600.

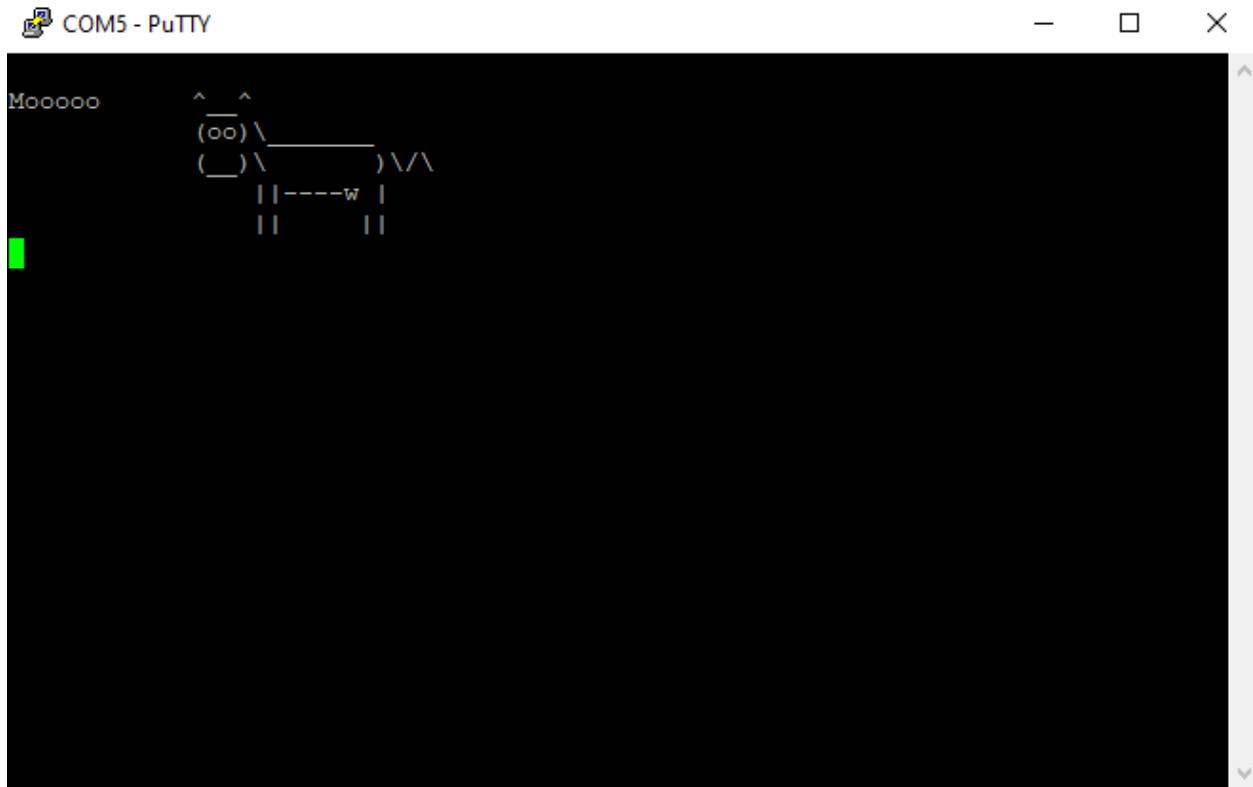


Figure 1.3

Included below is a screenshot of the PuTTY terminal with the program using a baud rate of 115200. No major changes were needed to make the program work.

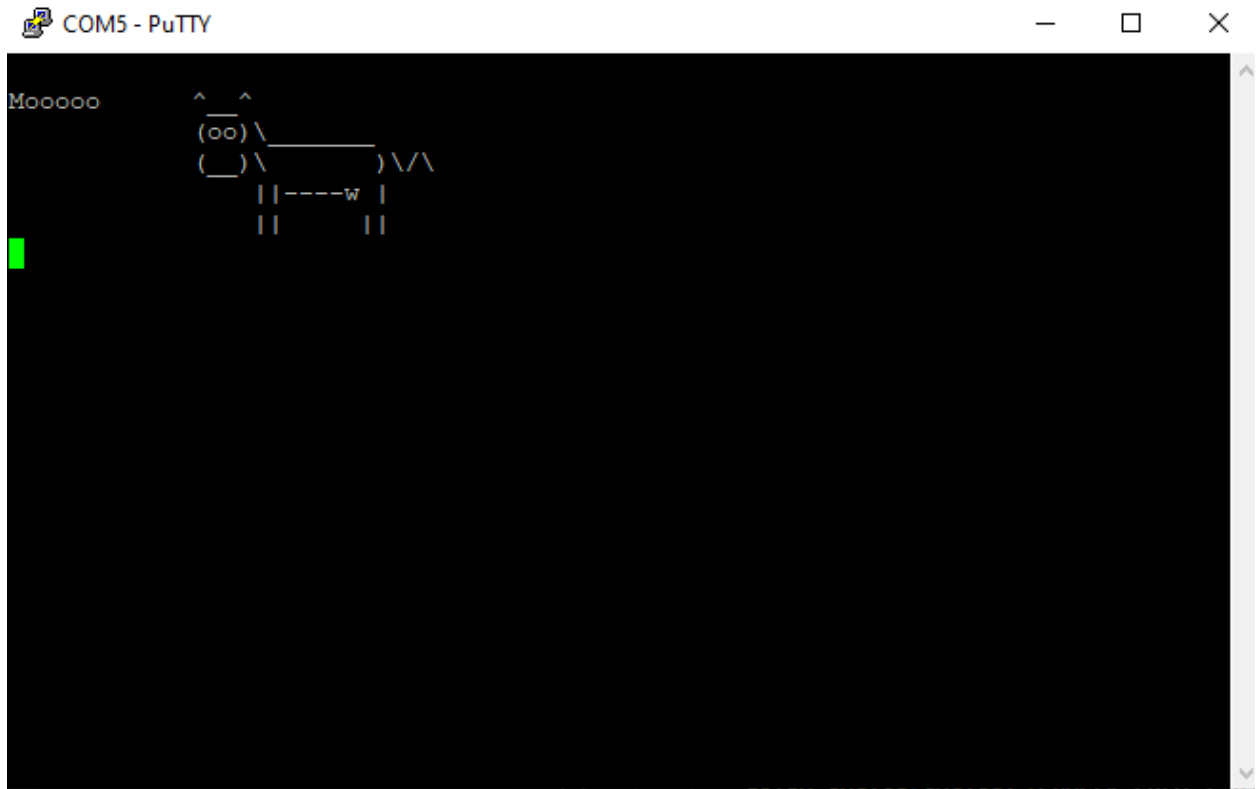


Figure 1.4

Exercise 2 – EnergyTrace

In this exercise, we will use EnergyTrace to measure a display an application's energy profile. This can be used to help us optimize power consumption of the system. The circuitry that supports EnergyTrace is on the debug probe side of the LaunchPad and it measures the current of the MCU in real time.

To use EnergyTrace, CCs v12.8 had to be installed.

Exercise 2.1 – Simple Checksum

In this exercise, we compute the CRC-32 checksum using different methods and measure the total energy consumption required. Using the files provided and using the procedure listed in the lab manual, we made the following table and bar graphs of the data.

	Energy 1	Mean Current 1	Energy 2	Mean Current 2	Energy 3	Mean Current 3	Average Energy	Average Mean Current
SW method	2760.179 mJ	12.2109 mA	2761.697 mJ	12.2217 mA	2758.043 mJ	12.2220 mA	2759.97 mJ	12.2182 mA
HW method	606.112 mJ	12.2476 mA	605.571 mJ	12.2366 mA	608.266 mJ	12.2332 mA	606.65 mJ	12.2391 mA
DMA method	85.704 mJ	12.2649 mA	88.594 mJ	12.2672 mA	88.515 mJ	12.2618 mA	87.60 mJ	12.2646 mA
DMA LPM method	73.367 mJ	11.3752 mA	73.317 mJ	11.3675 mA	74.214 mJ	11.3669 mA	73.63 mJ	11.3699 mA

Table 2.1

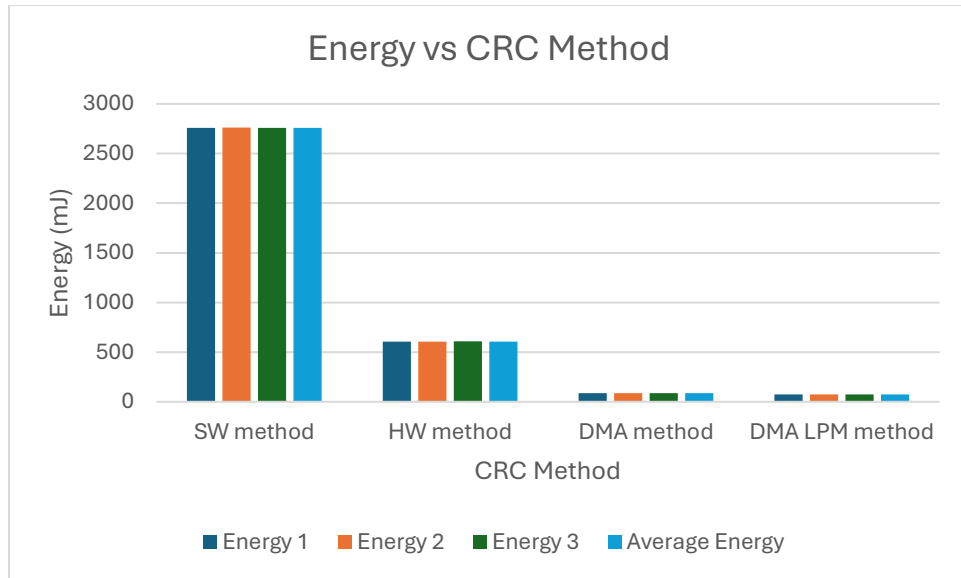


Figure 2.1

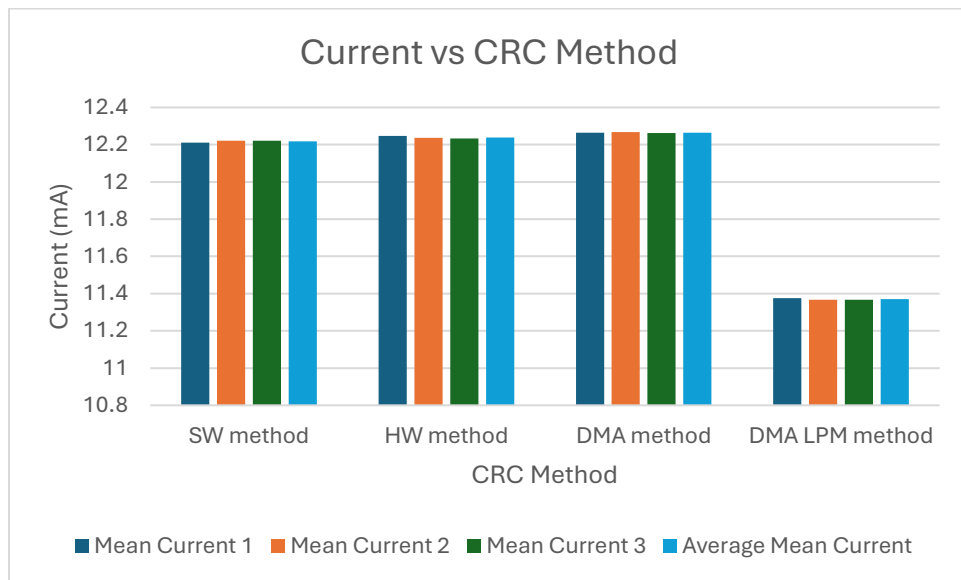


Figure 2.2

The energy graphs show that the software method uses the most energy while using similar current to the hardware and DMA methods. The higher energy usage is most likely due to the software method taking longer to compute the CRC compared to the hardware and DMA methods. The low power mode of the DMA method uses even less power compared to the DMA method. Since both methods use the same principles to calculate the CRC the low power mode method likely uses less energy since it uses almost 1 mA less current than the normal DMA method.

Exercise 2.2 – Shutdown LPM

In this exercise, we will compare the energy consumption of two different LED blinking implementations. One will use polling to check if S1 has been pressed to blink the LED while the other will use Shutdown Mode to shutdown the MCU while it is idling. The data collected of the two different methods is included in the table below.

	Energy 1	Mean Current 1	Energy 2	Mean Current 2	Energy 3	Mean Current 3	Average Energy	Average Mean Current
Shutdown mode	1.293 mJ	0.7477 mA	1.313 mJ	0.7795 mA	1.013 mJ	0.6432 mA	1.206 mJ	0.7235 mA
Run mode	75.346 mJ	2.2830 mA	75.838 mJ	2.2979 mA	76.199 mJ	2.3088 mA	75.794 mJ	2.2966 mA

Table 2.2

The energy savings can be calculated by using the following formula.

$$\% \text{Energy saved} = \frac{\text{Run mode avg} - \text{Shutdown mode avg}}{\text{Run mode avg}} \times 100$$

Plugging in the numbers from the table results in an energy savings of ~98.4%.

Appendix A – MCU Source Code

echo.c

```
#include "ti_msp_dl_config.h"
```

```
int main(void)
```

```
{
```

```
    SYSCFG_DL_init();
```

```
    static const DL_UART_Main_ClockConfig gUART_0ClockConfig = {  
        .clockSel = DL_UART_MAIN_CLOCK_BUSCLK,  
        .divideRatio = DL_UART_MAIN_CLOCK_DIVIDE_RATIO_1  
    };
```

```
    static const DL_UART_Main_Config gUART_0Config = {  
        .mode = DL_UART_MAIN_MODE_NORMAL,  
        .direction = DL_UART_MAIN_DIRECTION_TX_RX,  
        .flowControl = DL_UART_MAIN_FLOW_CONTROL_NONE,  
        .parity = DL_UART_MAIN_PARITY_NONE,  
        .wordLength = DL_UART_MAIN_WORD_LENGTH_8_BITS,  
        .stopBits = DL_UART_MAIN_STOP_BITS_ONE  
    };
```

```
    DL_UART_Main_reset(UART0);
```

```
    DL_UART_Main_enablePower(UART0);
```

```
    delay_cycles(POWER_STARTUP_DELAY);
```

```
    DL_UART_Main_setClockConfig(UART0, (DL_UART_Main_ClockConfig *)  
        &gUART_0ClockConfig);
```

```
    DL_UART_Main_init(UART0, (DL_UART_Main_Config *) &gUART_0Config);
```

```
    DL_UART_Main_setOversampling(UART0,  
DL_UART_OVERSAMPLING_RATE_16X);  
  
    DL_UART_Main_setBaudRateDivisor(UART0, 17, 23);  
  
    DL_UART_Main_enable(UART0);  
  
    DL_GPIO_initPeripheralOutputFunction(IOMUX_PINCM21,  
IOMUX_PINCM21_PF_UART0_TX);  
  
    DL_GPIO_initPeripheralInputFunction(IOMUX_PINCM22,  
IOMUX_PINCM22_PF_UART0_RX);  
  
  
    while (1) {  
        if (!DL_UART_isRXFIFOEmpty(UART0)) {  
            DL_UART_Main_transmitData(UART0,  
DL_UART_Main_receiveData(UART0));  
        }  
    }  
}
```

message.c

```
#include "ti_msp_dl_config.h"

void send_message(char* msg) {
    int i = 0;
    while (msg[i] != 0x00) {
        if (!DL_UART_isTXFIFOFull(UART0)) {
            DL_UART_Main_transmitData(UART0, msg[i]);
            i++;
        }
    }
}

int main(void)
{
    SYSCFG_DL_init();

    static const DL_UART_Main_ClockConfig gUART_0ClockConfig = {
        .clockSel = DL_UART_MAIN_CLOCK_BUSCLK,
        .divideRatio = DL_UART_MAIN_CLOCK_DIVIDE_RATIO_1
    };
    static const DL_UART_Main_Config gUART_0Config = {
        .mode = DL_UART_MAIN_MODE_NORMAL,
        .direction = DL_UART_MAIN_DIRECTION_TX_RX,
        .flowControl = DL_UART_MAIN_FLOW_CONTROL_NONE,
        .parity = DL_UART_MAIN_PARITY_NONE,
        .wordLength = DL_UART_MAIN_WORD_LENGTH_8_BITS,
```

```

        .stopBits = DL_UART_MAIN_STOP_BITS_ONE
    };

    DL_UART_Main_reset(UART0);
    DL_UART_Main_enablePower(UART0);
    delay_cycles(POWER_STARTUP_DELAY);
    DL_UART_Main_setClockConfig(UART0, (DL_UART_Main_ClockConfig *)
    &gUART_0ClockConfig);
    DL_UART_Main_init(UART0, (DL_UART_Main_Config *) &gUART_0Config);
    DL_UART_Main_setOversampling(UART0,
DL_UART_OVERSAMPLING_RATE_16X);

    DL_UART_Main_setBaudRateDivisor(UART0, 17, 23);

    DL_UART_Main_enable(UART0);

    DL_GPIO_initPeripheralOutputFunction(IOMUX_PINCM21,
IOMUX_PINCM21_PF_UART0_TX);

    DL_GPIO_initPeripheralInputFunction(IOMUX_PINCM22,
IOMUX_PINCM22_PF_UART0_RX);


    static char something[] =

        {0x0d,0x0a,0x4d,0x6f,0x6f,0x6f,0x6f,0x6f,0x20,0x20,0x20,0x20,0x20,
0x20,0x5e,0x5f,0x5f,0x5e,0x0d,0x0a,0x20,0x20,

        0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x28,0x6f,0x6f,0
x29,0x5c,0x5f,0x5f,0x5f,0x5f,0x5f,0x5f,0x5f,

        0x0d,0x0a,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0
x20,0x28,0x5f,0x5f,0x29,0x5c,0x20,0x20,0x20,

        0x20,0x20,0x20,0x20,0x29,0x5c,0x2f,0x5c,0x0d,0x0a,0x20,0x20,0x20,0
x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,

        0x20,0x20,0x20,0x20,0x7c,0x7c,0x2d,0x2d,0x2d,0x2d,0x77,0x20,0x7c,0
xd,0x0a,0x20,0x20,0x20,0x20,0x20,0x20,0x20,

        0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x7c,0x7c,0x20,0x20,0
x20,0x20,0x20,0x7c,0x7c,0x0d,0x0a,0x00};

```

```
send_message(something);
```

```
while (1) {  
    if (!DL_UART_isRXFIFOEmpty(UART0)) {  
        DL_UART_Main_transmitData(UART0,  
DL_UART_Main_receiveData(UART0));  
    }  
}  
}
```

emulate.c

```
#include "ti_msp_dl_config.h"

#define BAUD_RATE 9600

void send_message(char* msg) {
    int count = 0;

    DL_GPIO_enablePower(GPIOA);
    DL_GPIO_initDigitalOutput(IOMUX_PINCM21);    //UART TX
    DL_GPIO_enableOutput(GPIOA, DL_GPIO_PIN_10);
    DL_GPIO_initDigitalInput(IOMUX_PINCM22);    //UART RX

    DL_GPIO_setPins(GPIOA, DL_GPIO_PIN_10);
    delay_cycles(3200000);

    while (msg[count] != 0x00) {
        char data = msg[count];

        DL_GPIO_clearPins(GPIOA, DL_GPIO_PIN_10);
        delay_cycles((CPUCLK_FREQ / BAUD_RATE));
        for (int i = 0; i < 8; i++) {
            if (data & 1) {
                DL_GPIO_setPins(GPIOA, DL_GPIO_PIN_10);
                delay_cycles(CPUCLK_FREQ / BAUD_RATE);
            }
            else {
```

```

        DL_GPIO_clearPins(GPIOA, DL_GPIO_PIN_10);
        delay_cycles(CPUCLK_FREQ / BAUD_RATE);
    }
    data >>= 1;
}
DL_GPIO_setPins(GPIOA, DL_GPIO_PIN_10);
delay_cycles(CPUCLK_FREQ / BAUD_RATE);
count++;
}
}

int main(void)
{
    SYSCFG_DL_init();

    static char something[] =
        {0x0d,0x0a,0x4d,0x6f,0x6f,0x6f,0x6f,0x6f,0x20,0x20,0x20,0x20,0x20,
        0x20,0x5e,0x5f,0x5f,0x5e,0x0d,0x0a,0x20,0x20,
        0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x28,0x6f,0x6f,0
        x29,0x5c,0x5f,0x5f,0x5f,0x5f,0x5f,0x5f,0x5f,
        0x0d,0x0a,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0
        x20,0x28,0x5f,0x5f,0x29,0x5c,0x20,0x20,0x20,
        0x20,0x20,0x20,0x20,0x29,0x5c,0x2f,0x5c,0x0d,0x0a,0x20,0x20,0x20,0
        x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,
        0x20,0x20,0x20,0x20,0x7c,0x7c,0x2d,0x2d,0x2d,0x2d,0x77,0x20,0x7c,0
        xd,0x0a,0x20,0x20,0x20,0x20,0x20,0x20,0x20,
        0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x7c,0x7c,0x20,0x20,0
        x20,0x20,0x20,0x7c,0x7c,0x0d,0x0a,0x00};

    send_message(something);

```

```
        while (1) {  
        }  
}
```

lpm.c

```
/* DriverLib Includes */  
  
#include "ti_msp_dl_config.h"  
  
  
/* Standard Includes */  
#include <stdint.h>  
#include <stdbool.h>  
#include <stdio.h>  
  
#define USE_POLLING  
  
// Set up GPIO output pins  
void setup_LEDs(bool set_state)  
{  
    // LED  
    DL_GPIO_initDigitalOutput(IOMUX_PINCM57); // For PB26, red LED in  
LED2  
    DL_GPIO_initDigitalOutput(IOMUX_PINCM58); // For PB27, green LED  
in LED2  
    DL_GPIO_initDigitalOutput(IOMUX_PINCM50); // For PB22, blue LED in  
LED2  
  
    DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_26); // PB26, red LED  
    DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_27); // PB27, green LED
```

```

DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_22); // PB22, blue LED

if (set_state) // Do not set to known states if returning from
LPM. Need to preserve them
{
    DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_26);
    DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_27);
    DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_22);
}
}

// Set up for push buttons
void setup_push_buttons()
{
    DL_GPIO_initDigitalInput(IOMUX_PINCM49); // For PB21, push button
S2

    DL_GPIO_initDigitalInput(IOMUX_PINCM40); // For PA18, push button
S1

    DL_GPIO_initDigitalInputFeatures(IOMUX_PINCM49,
DL_GPIO_INVERSION_DISABLE, DL_GPIO_RESISTOR_PULL_UP,
DL_GPIO_HYSTERESIS_DISABLE, DL_GPIO_WAKEUP_DISABLE); //S2

#ifdef USE_POLLING
    DL_GPIO_initDigitalInputFeatures(IOMUX_PINCM40,
DL_GPIO_INVERSION_DISABLE, DL_GPIO_RESISTOR_NONE,
DL_GPIO_HYSTERESIS_DISABLE, DL_GPIO_WAKEUP_DISABLE); // S1

    return;
#else

```

```
    DL_GPIO_initDigitalInputFeatures(IOMUX_PINCM49,  
DL_GPIO_INVERSION_DISABLE, DL_GPIO_RESISTOR_PULL_UP,  
DL_GPIO_HYSTERESIS_DISABLE, DL_GPIO_WAKEUP_DISABLE); //S2
```

```
    DL_GPIO_initDigitalInputFeatures(IOMUX_PINCM40,  
DL_GPIO_INVERSION_DISABLE, DL_GPIO_RESISTOR_NONE,  
DL_GPIO_HYSTERESIS_DISABLE, DL_GPIO_WAKEUP_ON_1); // S1
```

```
#endif
```

```
}
```

```
// Blink briefly an LED
```

```
void blink_LED(GPIO_Regs * port, uint32_t pin)
```

```
{
```

```
    DL_GPIO_setPins(port, pin);
```

```
    delay_cycles(CPUCLK_FREQ / 100); // 10 ms at 32 MHz
```

```
    DL_GPIO_clearPins(port, pin);
```

```
}
```

```
// Function if need to stay in Run mode for better communication with  
the debugger
```

```
void check_to_stay_in_Run_Mode()
```

```
{
```

```
    if (DL_GPIO_readPins(GPIOB, DL_GPIO_PIN_21) == 0) // S2 being  
pressed while starting the program
```

```
    {
```

```
        while (1) // Blink the green LED to indicate it is in Run mode
```

```
        {
```

```
            DL_GPIO_togglePins(GPIOB, DL_GPIO_PIN_27);
```

```
            delay_cycles(CPUCLK_FREQ / 5);
```

```
        }
```

```

    }
}

int main(void)
{
    volatile DL_SYSCCTL_RESET_CAUSE resetCause;

    SYSCFG_DL_init();

    resetCause = DL_SYSCCTL_getResetCause();

    if (DL_SYSCCTL_RESET_CAUSE_BOR_WAKE_FROM_SHUTDOWN == resetCause) //
Woken up from a SHUTDOWN state
    {
        setup_LEDs(false);
        setup_push_buttons();
        DL_SYSCCTL_releaseShutdownIO();
        DL_GPIO_disableWakeUp(IOMUX_PINCM40);
        check_to_stay_in_Run_Mode();

        if (DL_GPIO_readPins(GPIOA, DL_GPIO_PIN_18) != 0) // S1 is
pressed
        {
            blink_LED(GPIOB, DL_GPIO_PIN_22); // Blink blue LED

            while (DL_GPIO_readPins(GPIOA, DL_GPIO_PIN_18) != 0) {} //
Wait until S1 is released
        }

        DL_SYSCCTL_setPowerPolicySHUTDOWN();
    }
}

```

```

    DL_GPIO_enableWakeUp(IOMUX_PINCM40);
    while(1) // Go back to Shutdown Mode
    {
        __WFI();
    }
}

// Not coming back from Shutdown Mode
setup_LEDs(true);
setup_push_buttons();
check_to_stay_in_Run_Mode();

while(1)
{
#ifdef USE_POLLING
    if (DL_GPIO_readPins(GPIOA, DL_GPIO_PIN_18) != 0) // S1 is
pressed
    {
        blink_LED(GPIOB, DL_GPIO_PIN_22); // Blink blue LED
        while (DL_GPIO_readPins(GPIOA, DL_GPIO_PIN_18) != 0) {} //
Wait until S1 is released
    }

#else
    DL_SYSCCTL_setPowerPolicySHUTDOWN();
    {
        __WFI();

```

```
    }  
#endif  
    }  
}
```